503 HTTP Status Code in ASP.NET Core Web API

Home » ASP.NET Core Web API Tutorial

503 HTTP Status Code in ASP.NET Core Web API

In this article, I will discuss **How to Return a 503 Service Unavailable HTTP Status Code in an ASP.NET Core Web API**. Please read the article to the end to understand the complete process. Prerequisites: Visual Studio 2022 and .NET Core 6.0 or higher.

What is 503 HTTP Status Code?

The 503 HTTP Status Code stands for "Service Unavailable," which indicates that the web server is currently unable to handle the request. The server may be too busy or undergoing maintenance or upgrades. Consider the following scenarios where the server might return a 503 Service Unavailable response:

- **Server Overload:** The server is experiencing high traffic or resource limits, leading to a temporary inability to process requests at the moment. This can be caused by various factors, including:
  - **Resource Exhaustion:** The server's CPU, memory, or bandwidth is maxed out, causing it to slow down or become unresponsive. This happens when the server lacks the resources to handle the incoming requests efficiently.
  - **Application Issues:** Unexpected bugs or errors in the application code can cause the server to crash or become unresponsive, leading to a 503 error.
  - **Application Pool Issues:** In environments like IIS, issues with the application pool, such as crashes, stalls, or timeouts, can result in 503 errors.

How to Return 503 HTTP Status Code in ASP.NET Core Web API

In ASP.NET Core Web API, returning a 503 HTTP status code (Service Unavailable) can be done explicitly within your controllers using the StatusCode method of the ControllerBase class or by configuring middleware to handle specific exceptions. Let us provide your understanding with both approaches.

We will create a simple example to demonstrate how to return a 503 Status Code in an ASP.NET Core Web API. The classes and methods we will use in this article are part of the ASP.NET Core framework, so no additional libraries need be installed. Open Visual Studio 2022 and create a new ASP.NET Core Web API project.

Configure appsettings.json

Open the file **appsettings.json**. We will use this file to set whether the service is under maintenance. So, add the following key-value **appsettings.json** file and save it. Here, it is:

```
{
  "IsUnderMaintenance": false,
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  }
}
```

Returning 503 from a Controller Action using StatusCode Method

You can return a 503 status code directly from any action within your controllers by using the StatusCode method of the ControllerBase class. Here's an example of how to implement this in your controller:

```
namespace ReturnTypesExamples.Controllers;
[ApiController]
[Route("api/[controller]")]
public class ExampleController : ControllerBase
{
    private readonly ILogger _logger;
    private bool _isUnderMaintenance;
    public ExampleController(ILogger logger, IConfiguration configuration)
    {
        _logger = logger;
        _isUnderMaintenance = configuration["IsUnderMaintenance"] ?
            false;
        // If configuration is not set, default to false.
        _isUnderMaintenance = _isUnderMaintenance ?
            _configuration.GetValue<bool>("IsUnderMaintenance", false) ?
            false;
        _logger.LogInfo("Controller initialized. Maintenance mode: {0}",
            _isUnderMaintenance);
    }
    [HttpGet("service-status")]
    public IActionResult GetServiceStatus()
    {
        _logger.LogInfo("Checking service status.");
        if (_isUnderMaintenance)
        {
            _logger.LogWarning("Service is currently under maintenance.");
            return StatusCode(503, "Service is currently under maintenance. Please try again later.");
        }
        _logger.LogInfo("Service is operational.");
        return Ok("Service is operational.");
    }
    [HttpGet("toggle-maintenance/{enable:bool}")]
    public IActionResult ToggleMaintenance(bool enable)
    {
        _logger.LogInfo("Toggling maintenance mode to {0}", enable);
        _isUnderMaintenance = enable;
        _logger.LogInfo("Maintenance mode is now {0}", _isUnderMaintenance);
        return Ok($"Maintenance mode {(enable ? "enabled" : "disabled")}");
    }
    [HttpGet("error")]
    public IActionResult ThrowException()
    {
        _logger.LogInfo("Throwing an exception.");
        throw new Exception("This is a test exception.");
    }
    [HttpGet("health")]
    public IActionResult GetHealth()
    {
        _logger.LogInfo("Checking health.");
        return Ok("Healthy");
    }
    [HttpGet("version")]
    public IActionResult GetVersion()
    {
        _logger.LogInfo("Checking version.");
        return Ok("1.0.0");
    }
    [HttpGet("time")]
    public IActionResult GetServerTime()
    {
        _logger.LogInfo("Checking server time.");
        return Ok(DateTime.Now);
    }
    [HttpGet("echo/{message}")]
    public IActionResult Echo(string message)
    {
        _logger.LogInfo("Echoing message: {0}", message);
        return Ok($"Echo: {message}");
    }
    [HttpGet("random")]
    public IActionResult GetRandomNumber()
    {
        _logger.LogInfo("Generating random number.");
        var random = new Random();
        int randomNumber = random.Next();
        return Ok($"Random number: {randomNumber}");
    }
    [HttpGet("divide/{a:int}/{b:int}")]
    public IActionResult Divide(int a, int b)
    {
        _logger.LogInfo("Dividing {0} by {1}", a, b);
        if (b == 0)
        {
            _logger.LogWarning("Division by zero attempted.");
            return BadRequest("Cannot divide by zero.");
        }
        int result = a / b;
        _logger.LogInfo("Result: {0}", result);
        return Ok($"Result: {result}");
    }
    [HttpGet("greet/{name}")]
    public IActionResult Greet(string name)
    {
        _logger.LogInfo("Greeting {0}", name);
        return Ok($"Hello, {name}!");
    }
    [HttpGet("status/{code:int}")]
    public IActionResult GetStatusCode(int code)
    {
        _logger.LogInfo("Returning status code {0}", code);
        return StatusCode(code, $"Status code {code}");
    }
    [HttpGet("log-test")]
    public IActionResult LogTest()
    {
        _logger.LogInfo("Logging test.");
        _logger.LogWarning("Warning test.");
        _logger.LogError("Error test.");
        return Ok("Logging test completed.");
    }
    [HttpGet("uptime")]
    public IActionResult GetUptime()
    {
        _logger.LogInfo("Checking uptime.");
        var uptime = DateTime.Now - Process.GetProcessById(Process.CurrentProcessId).StartTime;
        return Ok($"Uptime: {uptime.Days}.{uptime.Hours}:{uptime.Minutes}:{uptime.Seconds}");
    }
    [HttpGet("memory")]
    public IActionResult GetMemoryUsage()
    {
        _logger.LogInfo("Checking memory usage.");
        var memoryUsage = GC.GetTotalMemory(false);
        return Ok($"Memory usage: {memoryUsage} bytes");
    }
    [HttpGet("environment")]
    public IActionResult GetEnvironment()
    {
        _logger.LogInfo("Checking environment.");
        var environment = Environment.GetEnvironmentVariables();
        return Ok(environment);
    }
    [HttpGet("config/{key}")]
    public IActionResult GetConfigValue(string key)
    {
        _logger.LogInfo("Checking config value for key: {0}", key);
        var value = _configuration[key];
        return Ok($"Config value for {key}: {value}");
    }
    [HttpGet("log-level/{level}")]
    public IActionResult SetLogLevel(string level)
    {
        _logger.LogInfo("Setting log level to {0}", level);
        // This is just a placeholder, actual implementation would depend on your logging framework
        return Ok($"Log level set to {level}");
    }
    [HttpGet("random-error")]
    public IActionResult GetRandomError()
    {
        _logger.LogInfo("Generating random error.");
        var random = new Random();
        int errorCode = random.Next(400, 500);
        return StatusCode(errorCode, $"Random error {errorCode}");
    }
    [HttpGet("slow")]
    public IActionResult GetSlowResponse()
    {
        _logger.LogInfo("Starting slow response.");
        System.Threading.Thread.Sleep(5000);
        _logger.LogInfo("Finishing slow response.");
        return Ok("Slow response completed.");
    }
    [HttpGet("cpu")]
    public IActionResult GetCpuUsage()
    {
        _logger.LogInfo("Checking CPU usage.");
        var cpuUsage = Process.GetProcessById(Process.CurrentProcessId).TotalProcessorTime;
        return Ok($"CPU usage: {cpuUsage}");
    }
    [HttpGet("threads")]
    public IActionResult GetThreadCount()
    {
        _logger.LogInfo("Checking thread count.");
        var threadCount = Process.GetProcessById(Process.CurrentProcessId).Threads.Count;
        return Ok($"Thread count: {threadCount}");
    }
    [HttpGet("gc")]
    public IActionResult GetGCInfo()
    {
        _logger.LogInfo("Checking GC info.");
        var gcInfo = GC.GetGCMemoryInfo();
        return Ok($"GC info: {gcInfo}");
    }
    [HttpGet("dns/{host}")]
    public IActionResult GetDnsInfo(string host)
    {
        _logger.LogInfo("Checking DNS info for host: {0}", host);
        var dnsInfo = System.Net.Dns.GetHostEntry(host);
        return Ok($"DNS info for {host}: {dnsInfo.HostName}");
    }
    [HttpGet("ping/{host}")]
    public IActionResult Ping(string host)
    {
        _logger.LogInfo("Pinging host: {0}", host);
        var ping = new System.Net.NetworkInformation.Ping();
        var reply = ping.Send(host);
        return Ok($"Ping {host}: {reply.Status}");
    }
}
```

Now, run the application and make a request to the above endpoint. You should receive a 503 Service Unavailable response with the message "Service is currently under maintenance. Please try again later."

Implement a Custom Middleware Component to Return 503 Status Code

If you want to return a 503 status code based on specific conditions, such as maintenance mode, it's best practice to implement a custom middleware. This approach centralizes the logic instead of scattering it across various controllers. Create a new folder named **Middlewares** in your project. Inside this folder, add a new class file named **ServiceUnavailableMiddleware.cs**. This middleware will check if the application is in maintenance mode and return a 503 Service Unavailable response. The following code is a simplified example; in a real-world scenario, you'd typically read this from a configuration file.

```
namespace ReturnTypesExamples.Middlewares;
public class ServiceUnavailableMiddleware
{
    private RequestDelegate _next;
    private bool _isUnderMaintenance;
    public ServiceUnavailableMiddleware(RequestDelegate next, IConfiguration configuration)
    {
        _next = next;
        _isUnderMaintenance = configuration["IsUnderMaintenance"] ?
            false;
        // If configuration is not set, default to false.
        _isUnderMaintenance = _isUnderMaintenance ?
            _configuration.GetValue<bool>("IsUnderMaintenance", false) ?
            false;
        _logger.LogInfo("Middleware initialized. Maintenance mode: {0}",
            _isUnderMaintenance);
    }
    public Task Invoke(HttpContext context)
    {
        if (_isUnderMaintenance)
        {
            _logger.LogWarning("Service is currently under maintenance.");
            context.Response.StatusCode = 503;
            context.Response.ContentType = "text/plain";
            return context.Response.WriteAsync("Service is currently under maintenance. Please try again later.");
        }
        _logger.LogInfo("Service is operational.");
        return _next(context);
    }
}
```

Now, run the application and make a request to the above endpoint. You should receive a 503 Service Unavailable response with the message "Service is currently under maintenance. Please try again later."

Implementing Polly Logic to Handle the 503 Service Unavailable

Let's understand how to implement Polly logic to handle 503 Service Unavailable status code involves implementing retry logic in our application. This can be done using various libraries and techniques. One popular library for handling retries in .NET is Polly.

Polly is a .NET resilience and transient-fault-handling library that allows developers to express policies such as retries, fallbacks, and circuit breakers in a fluent and thread-safe manner. We can use Polly to handle 503 status codes when calling an external service or an HTTP API. Here's a simple example:

Step 1: Install Polly

First, we need to install Polly. You can do this using the Package Manager Console by executing the following command:

```
install-Package Polly
install-Package Microsoft.Extensions.Http.Polly
```

Step 2: Define the Retry Policy

Define a retry policy using Polly. You can configure it to retry the request a certain number of times with an exponential backoff delay. This ensures that the system doesn't overwhelm the service with the frequent retries. The retry policy will handle the 503 status code. The following code is a simplified example, so please read the comments inline for a better understanding.

```
using Polly;
using Polly.Extensions.Http;
// Define a retry policy.
var retryPolicy = Policy
    .HandleResult(r => r.StatusCode == 503)
    .WaitAndRetryAsync(
        retryCount: 3,
        sleepDurationProvider: (retryAttempt, outcome, context) =>
            TimeSpan.FromSeconds(Math.Pow(2, retryAttempt)),
        onRetryAsync: (outcome, timespan, retryAttempt, context) =>
            {
                var logger = context.GetLogger();
                logger?.LogWarning("Retry {0} after {1} seconds for 503 Service Unavailable.",
                    retryAttempt, timespan.TotalSeconds);
                return Task.CompleteTask();
            });
// Register the policy.
var serviceCollection = new ServiceCollection();
serviceCollection.AddHttpClient("MyClient")
    .AddPolicyHandler(retryPolicy);
// Build the service provider.
var serviceProvider = serviceCollection.BuildServiceProvider();
// Get the HttpClient.
var client = serviceProvider.GetHttpClient("MyClient");
// Make a request.
var response = client.GetAsync("http://localhost:5000/api/Example/service-status").Result;
// Check the response.
if (response.StatusCode == 503)
{
    Console.WriteLine("Service is temporarily unavailable.");
}
else
{
    Console.WriteLine("Service is operational.");
}
```

Customizing Retry Logic

You can customize the retry logic to suit your needs, such as:

- **Retry Count:** Change the number of retries.
- **Backoff Strategy:** Use a fixed delay instead of exponential backoff.
- **Logging:** Log retry attempts for better visibility into the retry condition.

Step 3: Configure HttpClient to Use the Policy

Ensure that your HttpClient is configured to use the policy. In the Program.cs file, modify the Program.cs class file as follows. This ensures that the HttpClient is configured to use the retry policy. Now, you can use the client for HttpClient and Microsoft.Extensions.Http.Polly will automatically handle the retries.

```
namespace ReturnTypesExamples;
public class Program
{
    public static void Main(string[] args)
    {
        var builder = WebApplication.CreateBuilder(args);
        // Add services to the container.
        builder.Services.AddControllers();
        // Learn more about configuring Swagger/OpenAPI at https://aka.ms/aspnetcore/swashbuckle
        builder.Services.AddEndpointsApiExplorer();
        builder.Services.AddSwaggerGen();
        // Register the policy.
        var serviceCollection = new ServiceCollection();
        serviceCollection.AddHttpClient("MyClient")
            .AddPolicyHandler(retryPolicy);
        // Build the service provider.
        var serviceProvider = serviceCollection.BuildServiceProvider();
        // Get the HttpClient.
        var client = serviceProvider.GetHttpClient("MyClient");
        // Make a request.
        var response = client.GetAsync("http://localhost:5000/api/Example/service-status").Result;
        // Check the response.
        if (response.StatusCode == 503)
        {
            Console.WriteLine("Service is temporarily unavailable.");
        }
        else
        {
            Console.WriteLine("Service is operational.");
        }
        // Add services to the container.
        var app = builder.Build();
        // Configure the HTTP request pipeline.
        if (app.Environment.IsDevelopment())
        {
            app.UseSwagger();
            app.UseSwaggerUI();
        }
        app.UseHttpsRedirection();
        app.UseAuthorization();
        app.MapControllers();
        app.Run();
    }
}
```

Step 4: Implement the Service

Implement the service that returns the 503 status code. Create a new controller (MyController) in your service. So, create a class file named MyController.cs within the Models folder and then copy and paste the following code. The following code is self-explanatory so please read the comments inline to understand the code.

```
namespace ReturnTypesExamples.Controllers;
[ApiController]
[Route("api/[controller]")]
public class MyController : ControllerBase
{
    // Inject the logger.
    private ILogger _logger;
    // Constructor.
    public MyController(ILogger logger)
    {
        _logger = logger;
    }
    // This action will return a 503 status code.
    [HttpGet("service-status")]
    public IActionResult GetServiceStatus()
    {
        _logger.LogInfo("Checking service status.");
        // Simulate a maintenance mode.
        bool isUnderMaintenance = true;
        if (isUnderMaintenance)
        {
            _logger.LogWarning("Service is currently under maintenance.");
            return StatusCode(503, "Service is currently under maintenance. Please try again later.");
        }
        _logger.LogInfo("Service is operational.");
        return Ok("Service is operational.");
    }
}
```

Now access the service endpoint, and you should get 503 Service Unavailable HTTP Response.

How to Troubleshoot 503 HTTP Status Code in ASP.NET Core Web API:

To troubleshoot 503 HTTP Status Code in ASP.NET Core Web API, you need to check the following:

- **Check Server Load:** Monitor server performance metrics (CPU, memory, and network usage) to identify if the server is overloaded. Implement load balancing and auto-scaling if necessary to manage incoming requests. Use tools like Performance Monitor, Azure Monitor, or AWS CloudWatch to track server performance. Scale up or add more instances if needed.
- **Maintenance Mode:** Ensure maintenance mode is not inadvertently enabled. Check configuration files or environment variables to see if maintenance mode is set to true. Disable maintenance mode if it's not intended to be active.
- **Application Logs:** Check the application logs for any errors or warnings that might indicate issues. Use logging frameworks like Serilog, NLog, or built-in ASP.NET Core logging to capture detailed logs.
- **Dependencies:** Verify that all dependencies (databases, APIs, services) are functioning correctly. Check connection strings, API keys, and network connectivity.
- **Load Balancers:** Ensure load balancers are distributing traffic across healthy instances. Check for load balancers in configuration settings.

In the next article, I will discuss **How to Return 409 Conflict HTTP Status Code in ASP.NET Core Web API** with examples. In this article, I have tried my best to explain **How to Return a 503 Service Unavailable HTTP Status Code in ASP.NET Core Web API**. I hope you will enjoy the concepts discussed in this article. If you find this article helpful, please share it.

Previous Article: How to Return 409 Conflict HTTP Status Code in ASP.NET Core Web API
Next Article: How to Return 500 Internal Server Error HTTP Status Code in ASP.NET Core Web API

1 thought on "503 HTTP Status Code in ASP.NET Core Web API"

Lorem
January 18, 2025 at 10:00

Implementing Polly Logic to Handle the 503 Service Unavailable.
In Resource Not Found exceptions.

Reply

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name *
Email *
Website

Post Comment

About Us | Privacy Policy | Contact | ASP.NET Tutorials | Angular Tutorials | ASP.NET Core Blazor Tutorials | ASP.NET Core Tutorials | ASP.NET Web Forms Tutorials | ASP.NET Web API Tutorials | ASP.NET Tutorials | Excel Computing Tutorials | Microsoft Azure Tutorials | Microsoft Azure Functions Tutorials | Dotnet Interview Questions and Answers | Dotnet Interview Questions | Dotnet Interview Questions and Answers | Dotnet Tutorials | Dotnet Tutorials Interview Questions | Dotnet Tutorials Interview Questions and Answers | Dotnet Tutorials Interview Questions and Answers | Dotnet Tutorials Interview Questions and Answers